FinanzasU

Objetivos Específicos — Técnica SMART

Plataforma Web de Gestión Financiera Personal para Estudiantes Universitarios

Campo	Detalle
Proyecto	FinanzasU — Plataforma Web de Gestión Financiera
Contrato	FINU-2026-001
Cliente	FinanzasU S.A.S. — Juan Camilo Triana (Representante Legal)
Desarrollador	Rise — Deibyd Castillo (Representante Legal)
Stack tecnológico	React 18 + Supabase (PostgreSQL + Auth + RLS) + Tailwind CSS + Recharts + Git + GitHub
Metodología	Scrum — 5 sprints de 15 días (75 días en total)
Supervisor académico	Gustavo Adolfo Osorio — Universidad del Valle, Sede Zarzal · 2026

¿Qué es la técnica SMART?

La técnica SMART es un marco de referencia ampliamente utilizado en gestión de proyectos para redactar objetivos de forma estructurada, clara y evaluable. Cada letra representa un criterio que el objetivo debe cumplir para considerarse bien formulado.

Letra	Criterio	Pregunta clave	Ejemplo aplicado a FinanzasU
S	**Específico**	¿Qué quiero lograr? ¿Quién está involucrado?	Implementar el módulo CRUD de transacciones con filtros y exportación CSV
M	**Medible**	¿Cómo sabré que lo logré? ¿Qué indicadores usaré?	El 100% de las operaciones CRUD funcionan sin errores en Postman
A	**Alcanzable**	¿Es realista con los recursos y el tiempo disponibles?	4 devs + Supabase SDK + 15 días es suficiente para el módulo
R	**Relevante**	¿Aporta valor al proyecto y a los usuarios?	Sin transacciones no existe el resto del producto
T	**Temporal**	¿Cuándo debe estar completado?	Entrega completa al finalizar el Sprint 2 — día 30

Nota:
Los 6 objetivos específicos a continuación describen las grandes entregas del proyecto FinanzasU. Están alineados con los 5 sprints del contrato FINU-2026-001 y con las 26 historias

de usuario del Product Backlog. Cada objetivo desglosa los 5 criterios SMART en términos concretos del stack tecnológico elegido: React + Supabase + Tailwind + Recharts.

Objetivos Específicos del Proyecto

OE-01 · Diseñar una interfaz web intuitiva y responsive con React y Tailwind CSS

Enunciado del objetivo:
Diseñar e implementar una interfaz web intuitiva, accesible y responsive con React 18 y Tailwind CSS para la plataforma FinanzasU, que permita a estudiantes universitarios sin experiencia financiera registrar su primera transacción en menos de 3 minutos desde el primer login, validado con pruebas de usabilidad con mínimo 8 usuarios reales, y entregada al finalizar el Sprint 4 (día 60 del proyecto).

Criterio SMART	Descripción
S — Específico	La interfaz debe cubrir 7 páginas: Login, Register, Dashboard, Transacciones, Categorías, Presupuestos y Perfil. Desarrollada en React 18 con componentes funcionales, hooks y Tailwind CSS para estilos. Dirigida a estudiantes universitarios de 17 a 28 años sin experiencia financiera. Responsable: Dev 1 (Frontend Lead) y Dev 2 (JavaScript/hooks).
M — Medible	• El tiempo para registrar la primera transacción desde el login no supera 3 minutos en pruebas con usuarios reales. • El 80% de los 8+ testers completa el flujo completo sin asistencia del equipo. • La interfaz es funcional en 375px (móvil) y 1280px (escritorio) verificado con DevTools. • Todas las páginas obtienen un score de usabilidad ≥ 3.5/5 en la encuesta de satisfacción.
A — Alcanzable	El equipo Rise cuenta con Dev 1 dedicado a React/Tailwind y Dev 2 a hooks y lógica JS durante los 5 sprints. Tailwind CSS elimina la necesidad de escribir CSS personalizado, acelerando el desarrollo visual. El diseño se construye de forma incremental sprint a sprint partiendo de los mockups de Figma del Sprint 0.
R — Relevante	Una interfaz intuitiva es el factor principal de adopción. Si el diseño es complejo, los estudiantes abandonan la herramienta antes de ver su valor. Este objetivo responde directamente al problema documentado: las apps financieras existentes son difíciles de usar para el público objetivo universitario colombiano.
T — Temporal	• Sprint 1 (día 15): Login, Register, Layout y Navbar responsive. • Sprint 2 (día 30): Página de transacciones con tabla, filtros y formulario. • Sprint 3 (día 45): Páginas de categorías y presupuestos con barras de progreso. • Sprint 4 (día 60): Dashboard completo con contenedores de gráficas Recharts. • Sprint 5 (día 75): Correcciones de UX tras pruebas con usuarios reales.

OE-02 · Implementar el módulo de registro y gestión de transacciones con Supabase

Enunciado del objetivo:
Implementar un módulo funcional de registro de ingresos y gastos con operaciones CRUD completas (crear, leer, actualizar y eliminar), filtrado por mes, búsqueda por texto, paginación y exportación a CSV, consumiendo la base de datos PostgreSQL de Supabase con Row Level Security activo, operativo para todos los usuarios autenticados al finalizar el Sprint 2 (día 30 del proyecto).

Criterio SMART	Descripción
S — Específico	El módulo registra transacciones con: monto (DECIMAL > 0), tipo (ingreso/gasto), categoría, fecha y descripción opcional. Usa Supabase PostgreSQL con RLS para que cada usuario solo acceda a sus propios datos. Incluye historial mensual, filtros combinados y exportación CSV generada en el cliente. Responsables: Dev 3 (services + hooks) y Dev 4 (esquema SQL + políticas RLS).
M — Medible	• El 100% de las operaciones CRUD funcionan sin errores validadas con Postman. • Las políticas RLS impiden el acceso cruzado entre usuarios verificado con 2 cuentas. • El tiempo de respuesta de la consulta de listado con filtros no supera 1 segundo con 100 registros. • El archivo CSV exportado contiene encabezados correctos y todos los registros del mes visible.
A — Alcanzable	El módulo se construye sobre el esquema de Supabase diseñado en el Sprint 1. El SDK @supabase/supabase-js simplifica las consultas con filtros encadenados sin escribir SQL manual. Las 4 historias del Sprint 2 están dimensionadas para 15 días con el equipo completo de Rise.
R — Relevante	El registro de transacciones es la funcionalidad central de la app: sin ella no existe el resto del producto. Es la primera funcionalidad de valor tangible que el estudiante experimenta después del login, y la que genera el hábito de uso continuo.
T — Temporal	• Días 16–20: Esquema SQL, políticas RLS y transaccionesService.js. • Días 21–25: Hook useTransacciones.js y página Transacciones.jsx con CRUD. • Días 26–28: Filtros, paginación y exportación CSV. • Día 30: Entrega completa, probada y aprobada por EL CLIENTE.

OE-03 · Implementar el sistema de presupuesto mensual con alertas visuales automáticas

Enunciado del objetivo:
Implementar un sistema de presupuestos mensuales que permita al estudiante definir un límite de gasto por categoría en Supabase PostgreSQL y recibir alertas visuales automáticas en React cuando el consumo supere el 80% y el 100% del límite establecido, con porcentaje calculado mediante función RPC de Supabase, disponible al finalizar el Sprint 3 (día 45 del proyecto).

Criterio SMART	Descripción

S — Específico	El sistema crea, edita y elimina presupuestos por categoría y mes. Cada presupuesto muestra monto límite, monto gastado calculado con SUM() en Supabase RPC, porcentaje consumido y barra de progreso con tres estados: azul (0–79%), amarillo (80–99%) y rojo (100%+). Responsables: Dev 3 (presupuestosService + RPC) y Dev 4 (SQL de la función get_presupuestos_con_gasto).
M — Medible	• La función RPC get_presupuestos_con_gasto retorna el porcentaje correcto comparado con cálculo manual en 100% de los casos de prueba. • La alerta amarilla aparece visualmente en React cuando el porcentaje supera el 80%. • La alerta roja aparece correctamente al superar el 100%. • Los presupuestos se reordenan automáticamente (rojos primero) tras actualizar los datos.
A — Alcanzable	El módulo reutiliza la tabla transacciones ya existente del Sprint 2 para calcular gastos reales vía Supabase RPC. El porcentaje se calcula en PostgreSQL directamente (SUM condicional), lo que evita procesamiento en el cliente. Las 6 historias del Sprint 3 están dimensionadas para 15 días.
R — Relevante	La mayoría de los estudiantes no gastan de más intencionalmente — simplemente no tienen visibilidad de cuánto llevan acumulado. Las alertas automáticas dan esa visibilidad en tiempo real sin que el usuario deba calcular nada, respondiendo directamente al problema de falta de planificación financiera.
T — Temporal	• Días 31–35: Función SQL get_presupuestos_con_gasto + políticas RLS de presupuestos. • Días 36–40: presupuestosService.js + hook usePresupuestos.js. • Días 41–43: Página Presupuestos.jsx con tarjetas, barras de progreso y alertas. • Días 44–45: Página Categorias.jsx + hook useCategorias.js. • Día 45: Entrega completa y aprobada.

OE-04 · Desarrollar el dashboard de estadísticas visuales con Recharts y Supabase RPC

Enunciado del objetivo:

Desarrollar un dashboard interactivo con visualizaciones gráficas usando Recharts que consuma funciones RPC de Supabase PostgreSQL para mostrar: distribución de gastos por categoría (gráfica de torta), evolución de ingresos y gastos de los últimos 6 meses (gráfica de barras), balance neto histórico (gráfica de línea) y tarjetas de resumen del mes, filtrable por período, disponible al finalizar el Sprint 4 (día 60 del proyecto).

Criterio SMART	Descripción
S — Específico	El dashboard incluye: 4 tarjetas de resumen (ingresos, gastos, balance, N° transacciones), gráfica de torta con tooltips por categoría, gráfica de barras mixta con línea de balance, gráfica de línea histórica y top 5 categorías. Consume 5 funciones RPC de Supabase. Responsables: Dev 1 (Dashboard.jsx + Recharts), Dev 2 (useDashboard.js + fetch paralelo), Dev 3 (Edge Functions/RPC), Dev 4 (consultas SQL optimizadas).

M — Medible	• Las 3 gráficas Recharts se renderizan con datos reales de Supabase en el 100% de los casos. • El dashboard carga completamente en menos de 3 segundos con datos de 6 meses. • Al cambiar el selector de mes todas las tarjetas y gráficas se actualizan sin recargar la página. • Los tooltips de cada gráfica muestran el valor correcto al pasar el cursor.
A — Alcanzable	Recharts se instala vía npm y es nativa para React — no requiere configuración adicional. Las funciones RPC de Supabase centralizan la lógica de cálculo en PostgreSQL. El equipo construyó los módulos de transacciones y presupuestos en sprints anteriores, por lo que los datos ya están disponibles.
R — Relevante	El dashboard es la sección más motivadora para el usuario: ver visualmente en qué gasta su dinero genera el impacto educativo que justifica el proyecto. Sin visualizaciones, FinanzasU sería solo un libro contable digital sin valor diferencial sobre una hoja de cálculo.
T — Temporal	• Días 46–50: Funciones SQL RPC (get_resumen_mes, get_gastos_por_categoria, get_evolucion_mensual). • Días 51–54: dashboardService.js + hook useDashboard.js con fetch paralelo. • Días 55–58: Dashboard.jsx con Recharts (torta, barras, línea) y tarjetas de resumen. • Días 59–60: Top 5 categorías, estado de presupuestos y selector de mes funcional.

OE-05 · Validar la usabilidad de la plataforma con estudiantes universitarios reales

Enunciado del objetivo:

Realizar pruebas de usabilidad con un mínimo de 8 estudiantes universitarios de la Universidad del Valle sede Zarzal, aplicar una encuesta de satisfacción con escala 1–5, documentar el 100% de los bugs encontrados, corregir el 100% de los críticos y el 80% de los medios, y obtener un promedio de satisfacción general de al menos 3.5/5, todo completado antes del día 70 del proyecto.

Criterio SMART	Descripción
S — Específico	Las pruebas involucran estudiantes de 17–28 años que nunca usaron FinanzasU. Cada tester completa el flujo completo sin asistencia: registro → transacción → presupuesto → dashboard. Se mide satisfacción en 3 dimensiones: facilidad de uso, utilidad percibida y diseño visual. Coordinador: Dev 2 de Rise. Los bugs se registran con ID, descripción, pasos para reproducir y severidad (crítico, medio, bajo).
M — Medible	• Mínimo 8 testers completan el flujo completo sin intervención del equipo. • Promedio de satisfacción general ≥ 3.5/5 en la encuesta de Google Forms. • 100% de bugs críticos resueltos con commit descriptivo en GitHub antes del día 72. • 80% de bugs medios resueltos antes del día 72. • Reporte de pruebas documentado y entregado a EL CLIENTE.

A — Alcanzable	Los testers se reclutan del entorno universitario del mismo equipo. El Sprint 5 de 15 días está íntegramente dedicado a pruebas, correcciones y documentación. El mínimo de 8 testers es conservador y perfectamente viable en el entorno académico de la Universidad del Valle sede Zarzal.
R — Relevante	Las pruebas con usuarios reales son la única forma de validar que FinanzasU resuelve el problema identificado. Sin esta validación el proyecto sería un ejercicio técnico sin evidencia de utilidad real, y no cumpliría el criterio de aceptación global del Contrato FINU-2026-001.
T — Temporal	• Días 61–63: Plan de pruebas, guía de tareas para testers y encuesta en Google Forms. • Días 64–67: Sesiones de prueba con los 8+ estudiantes. • Día 68: Análisis de resultados y priorización de bugs por severidad. • Días 69–72: Corrección de bugs críticos y medios con prueba de regresión. • Días 73–75: Verificación final y entrega del reporte de pruebas a EL CLIENTE.

OE-06 · Entregar la documentación técnica completa del sistema

Enunciado del objetivo:

Generar y entregar la documentación técnica completa del sistema FinanzasU, incluyendo: diagrama entidad-relación actualizado de Supabase, documentación de todos los servicios y funciones RPC, manual de usuario ilustrado con mínimo 10 capturas de pantalla de la versión final en React, README con instrucciones de instalación verificadas en equipo limpio y documentación de políticas RLS, antes del día 75 del proyecto.

Criterio SMART	Descripción
S — Específico	La documentación cubre: (1) diagrama ER en /docs/diagrama-ER.png, (2) /docs/API.md con todos los servicios Supabase y funciones RPC, (3) manual de usuario PDF/Word con capturas de la app React, (4) README.md con instalación verificada (npm install + Supabase setup), (5) /docs/rls-policies.md con todas las políticas RLS. Responsable por capa: Dev 1 (capturas + manual), Dev 2 (README), Dev 3 (API.md), Dev 4 (diagrama ER + RLS).
M — Medible	• El README es seguido por un integrante no involucrado y el proyecto corre sin pasos adicionales. • El manual de usuario tiene mínimo 10 capturas de pantalla de la versión final de React. • Todos los servicios Supabase y funciones RPC están documentados con parámetros y respuestas. • El diagrama ER refleja el estado final del esquema sin discrepancias con Supabase Studio.
A — Alcanzable	La documentación se produce en paralelo con las correcciones del Sprint 5. Cada desarrollador documenta su capa específica. El tiempo estimado total es de 10–12 horas distribuidas entre los 4 devs. Las capturas del manual se toman de la app React en su versión final.
R — Relevante	La documentación es un requisito académico del curso y también garantiza que el proyecto puede ser retomado, escalado o evaluado por terceros.

	Específicamente, el diagrama ER y las políticas RLS documentadas permiten que EL CLIENTE mantenga el proyecto Supabase sin depender del equipo Rise.
T — Temporal	• Día 61: Diagrama ER final exportado y políticas RLS documentadas en /docs/rls-policies.md. • Días 62–68: API.md con servicios y RPC en paralelo con corrección de bugs. • Días 70–72: Manual de usuario con capturas finales de la app React. • Día 73: README verificado en equipo limpio. • Días 74–75: Revisión final de toda la documentación y entrega a EL CLIENTE.

Resumen General de Objetivos

La siguiente tabla consolida los 6 objetivos específicos con su sprint de entrega, indicador clave de medición y criterio de aceptación para EL CLIENTE.

ID	Objetivo	Sprint	Indicador principal	Entrega
OE-01	Interfaz web React + Tailwind CSS intuitiva y responsive	S1–S4	Tiempo de primera transacción < 3 min · 8+ testers	Día 60
OE-02	Módulo CRUD de transacciones con Supabase + RLS	S2	100% CRUD sin errores en Postman · RLS verificado	Día 30
OE-03	Presupuesto mensual con alertas 80%/100% via RPC	S3	Porcentaje correcto en 100% de casos · alertas visuales	Día 45
OE-04	Dashboard con Recharts consumiendo Supabase RPC	S4	3 gráficas renderizan datos reales en < 3 seg	Día 60
OE-05	Validación con 8+ usuarios reales (satisfacción ≥ 3.5/5)	S5	Promedio encuesta ≥ 3.5 · 100% bugs críticos resueltos	Día 70
OE-06	Documentación técnica completa (ER, API, manual, README)	S5	README verificado en equipo limpio · 10+ capturas	Día 75

Relación entre sprints y objetivos

Sprint	Período	OE que se avanza	Descripción del avance
Sprint 1	Días 1–15	OE-01 (parcial)	Login, Register, Layout y Navbar responsive en React/Tailwind

Sprint 2	Días 16–30	OE-01 (parcial) + OE-02	Transacciones.jsx + CRUD Supabase + filtros + paginación + CSV
Sprint 3	Días 31–45	OE-01 (parcial) + OE-03	Categorias.jsx + Presupuestos.jsx + RPC + alertas visuales
Sprint 4	Días 46–60	OE-01 (cierra) + OE-04	Dashboard.jsx + Recharts + 5 funciones RPC completas
Sprint 5	Días 61–75	OE-05 + OE-06	Pruebas reales + corrección bugs + documentación completa

Compromiso de las partes:

Los presentes objetivos específicos fueron revisados y acordados entre EL CLIENTE (FinanzasU S.A.S. — Juan Camilo Triana) y EL DESARROLLADOR (Rise — Deibyd Castillo) como parte del marco académico del proyecto. Su cumplimiento será evaluado en cada Sprint Review mediante el Acta de Aceptación firmada, conforme a lo establecido en el Contrato N° FINU-2026-001.